

A Brief Description on Pointers, References and Memory Management in Java and C++

By AKM Sabbir

Date: 08/12/2015

Introduction:

In this write up I am going to discuss about reference, pointers and memory management used in C++ and java. I am going to discuss about what are these and how do they work? Therefore I will discuss what are techniques used in programming language like C++ and java to access memory then I will give a brief description on those techniques. Then I will describe about the memory management for C++, java and garbage collection in java language.

Accessing memory:

There are possibly four ways in C++ to access data stored in memory. Such as pointers, using reference, value type variable and dynamic pointer such as iterators of standard template library. In java there are possibly two ways reference type and value type variables. When we pass a reference variable as a parameter in a function the reference variable itself is a pass by value but the object it points is a pass by reference. One example can make this thing clear,

```
Public void sum(SomeClass var_1){  
    Var_1 = new Object()  
}
```

Var_1 is an object reference referenced some class object named SomeClass. However inside class function sum it is now referencing to new object named Object. This operation does not effect at all to the old object of type SomeClass. However if we make any change to the object initially var_1 was pointing to then the affect is globally visible. Such as

```
Public void add_element(List l){  
    l.add(new item())  
}
```

This change to the object referenced by reference variable l is global. However there is no way in java to create the reference of value type such as integer, Boolean, byte, long, float. However we can wrap them up or decorate them with an object or there is another way just to create a single element array because arrays are pass by reference to get reference from them. There is an iterator interface in java that also can be used to cycle through the elements stored in a collection objects. It also enables programmer to obtain and remove elements from the container. Before we can access a collection through an iterator, we must obtain an iterator. Each of the different collection class provides an iterator() method that returns an iterator to the start of the collection. By using this iterator object, we can access each element in the collection, one element at a time.

In C++ we can access the memory by using pointers and references as following way,

```
Int x = 5;  
Int* points;  
Points = &x;
```

A pointer can point to nothing initially and we call it a bad pointer. It can be reassigned to point to any memory location. A void type pointer can be casted to any pointer type. On the other hand a reference type in C++ has to be assigned during its initialization. A reference is a kind of alternative name for a variable and it cannot be reassigned. There is no concept of bad reference in C++. However it can be made bad in some other way. There is also dynamic pointer named iterator used in standard template library of C++. Actually pointer itself is a kind of iterator because according to the definition of iterator in C++ which is, an iterator is an object that is pointing to some element in a range of elements and has the ability to iterate through the elements of that range using a set of operators such as increment operator and dereferencing operator *. There are different types of iterator in standard template library of C++ such as bidirectional iterator, forward iterator etc. However the advantage with stl iterator is a programmer does not need to worry about memory management. Any dynamic change in the data a container containing and an iterator is pointing to that container will immediately invalidate that iterator which is not true for pointers. We can easily cycle through the data contained in a container by using iterator even we can introduce changes. Container will be released and iterator will be invalidated as soon as they go out of scope.

Pointers in C++:

Definition: A **pointer** is a value that designates the address (i.e., the location in memory), of some value. Pointers are variables that hold a memory location. An example will clear this,

```
Int number = 5;
Int *numPtr = &number;
```

A simple integer variable currently holding a value is 5. This variable also plays the role of pointee for the pointer variable of the next statement. numPtr a pointer variable, the current value is a reference to a pointee number above. Now if we declare second pointer and make it pointed to the same pointee as following,

```
Int * numPtr2 = numPtr.
```

Then both the pointer will point to a same memory location. This is a kind of shallow copying/sharing. We could have used value type variables to get deep copying. In C++ when a pointer first allocated it does not have any pointee, the pointer is simply uninitialized and it is known as bad pointer. There is nothing automatic in C++ that will give a valid pointee to a pointer. It does not provide any runtime checking therefore following code is a serious runtime error in C++,

```
Void bad_pointer_name(){
    Int *pointers;
    *pointers = 42;
}
```

The behavior of above program execution is undefined. So a pointer in C++ generally follows the following rules

- A pointer stores the reference of a pointee. A pointee in turn stores something useful.
- Dereference operations on a pointer access its pointee. A pointer may only be dereferenced after it is been assigned to refer to a pointee. Most pointer related bugs involve violating this rule.
- Allocating a pointer through declaration does not automatically assign it to refer to a pointee. It's a separate operation that a pointer is assigned to refer to a pointee.
- Assignment between two pointers makes them refer to the same pointee which introduces sharing.

in C++ a pointer can point to a variable that is stored either in stack or heap. If it is stored in the heap then we have to be careful about the memory management. That is we have to make sure that we are de-allocating the stored memory otherwise that will be a resource loss. In terms of stacked data that is if a pointer points to a data that is stored in a local stack of a function in that case the memory will be de-allocated as soon as the variable storing the value will go out of scope of the local function and the pointer will be invalidated. Following example will make these things clear,

```
MyClass myClass;  
myClass.DoSomething();
```

First statement will instantiate a myClass object and allocate the myclass object into stack this happened compile time and not expensive. The advantage with this is stored object will be de-allocated as soon as the myClass object goes out of the scope. There is no chance of memory leakage here. However there is another way to instantiate the same object and perform operation on it. Such as,

```
MyClass *myClass;  
myClass = new MyClass()  
myClass -> DoSomething()  
delete myClass
```

this will allocate the myClass object into the heap a dynamic memory. The main differences are we are using asterisk(*) to declare a pointer variable. Then we are using **new** keyword of C++ to dynamically allocate the memory to a newly instantiated MyClass object. Then we have used arrow operator to access the dereferenced member function of that object. The most important thing is the last statement. We have to de-allocate the memory manually at the end of all operation and when we will no longer need the object. Otherwise it will create the memory leakage.

Pointer seem to be a first class object in C++, because it satisfies the following conditions,

- can be stored in variables and data structures
- can be passed as a parameter to a subroutine
- can be returned as the result of a subroutine
- can be constructed at runtime
- has intrinsic identity (independent of any given name)

However if we look into deep carefully we will find that If we are using pointers merely to 'point' to data, it will satisfy the requirements of being an FCO. However, if we use a pointer to 'hold' a data, then it can no longer be considered an FCO as its copy and assignment semantics do not work.

References in C++:

In C++ references provides the many same capabilities as pointers does. A reference is like a pointer object that can be used to refer to another object indirectly. A reference declaration essentially has a same syntactic structure as a pointer declaration. The difference is, a pointer declaration uses * operator while a reference declaration uses & operator. Such as,

```
Int val = 5;  
Int *ptr = &val.
```

Here ptr is a pointer object known as pointer to integer.

```
Int &ref = val.
```

Here ref is an object of type reference to int.

References are significant in C++ for some cases, consider the following case,

```
Enum Day{
    Saturday, Sunday, Monday.....
}
Day x;
```

Now if we apply ++x. it will not compile. We need to overload the ++ operator. Invoking the overloaded function should change the value in x. therefore declaring operator ++ with a parameter type of day, as in:

Day operator++(Day d) will not have the desired effect. This function passes its argument as a value therefore the function sees a copy of the argument not the argument itself. So in order to alter the value the parameter must be a pass by reference or pointer. But using pointer like Day* operator++(Day *d) will force us to use the operator in this way ++&x which is we don't want. The proper to overload ++ operator with a reference as a parameter type as in ,

```
day &operator++(day &d){
    d = (day)(d + 1);
    return d;
}
```

Using the function ++x will have the proper appearance as well as correct behavior. This is the only way in c++ to do this.

There is no rebinding of reference in C++. It means once a reference is bonded with an object it cannot be rebounded with a new object. There is no syntax for rebinding a reference in C++. It will be clarified with the following example,

```
Int &ri = integer;
```

Which bind ri to integer variable, then an assignment like below,

```
ri = j;
```

is not going to rebind ri with variable j, it will assign the value in j into the variable ri referencing that is in integer. This is not true for pointers because a pointer can point different objects during its lifetime. A reference can not be null in C++. A valid reference must refer to something however it may refer to some object which in turn can be null in C++. For instance a reference can refer to a null pointer. Then the reference will be a null reference. But technically, the error is not binding the reference but dereferencing the pointer which is null. Unlike pointer a reference cannot be declared constant in C++ but there is an indirect way to solve this problem in following way,

```
typedef int & ref_to_value;
ref_to_value const r = integer;
```

it seems C++ references are first class citizen. References also work with the polymorphism in C++ such as

```
struct Base { };
```

```

struct Derived : Base;

void foo(Base &);

int main() {
    Derived x;
    foo(x); // fine
}

```

Foo function can receive any base or derived reference parameter. Just like pointer to a function it is possible to have a reference to a function in C++,

```

void Foo(){
}

int main(){
    void(& func)() = Foo;
    func(); //::Foo();
}

```

References in Java:

In java a reference to an object or an array is simply some fixed-size value that refers to the object or array in some way. When we assign an object or array to a variable, we are actually setting the variable to hold a reference to that object or array. Similarly, when we pass an object or array to a method, what really happens is that the method is given a reference to the object or array through which it can manipulate the object or array. Now the question is why we need references to point to an object and arrays whereas primitive does not need that? Why is this extra bit of complexity? The answer to this question is,

all the Java primitive types have well-defined standard sizes, so all primitive values can be stored in a fixed amount of memory (between one and eight bytes, depending on the type). But classes and array types are composite types; objects and arrays contain other values, so they do not have a standard size, and they often require quite a bit more memory than eight bytes. For this reason, Java does not manipulate objects and arrays directly. Instead, it manipulates references to objects and arrays. Because Java handles objects and arrays by reference, classes and array types are known as reference types. In contrast, Java handles values of the primitive types directly, or by value. Unlike pointers java references cannot be manipulated. It cannot be incremented or decremented which is not entirely true for C++ references because if a reference in C++ refers to a pointer then pointer arithmetic operation on that pointer will eventually affect the reference that is referring to it. There are two main differences between java primitive types and reference types they are the way values are copied and the way they are compared for equality. Consider the following code that manipulate to primitive integer type,

```

Int x =30;
Int y = x;

```

Inside the virtual machine there will be two independent copies of the 32-bit integer 30. But this is not true for reference types,

```

Point p = new Point(x,y);
Point q = p;

```

Here q holds a copy of reference held in the variable p. but both point to same object. The same object can be accessible by using these two different references. Similar thing happens with the java arrays. We get exactly same effect when we use primitive type and reference type as a method parameter. In order to get the copy of objects we have to call the clone() method such as,

```
Point p = new Point(1,2); // p refers to one object
Point q = (Point) p.clone(); // q refers to a copy of that object
q.y = 42; // Modify the copied object, but not the original
int[] data = {1,2,3,4,5};
int[] copy = (int[]) data.clone();
```

Java only allows an object to be cloned if the object's class has explicitly declared itself to be cloneable by implementing the Cloneable interface. Arrays are always cloneable which is not true for all objects such as the point object used here. Another thing with clone method is it is implemented to create the shallow copy of an object and array. The copied object and array contains copies of all primitive values and references of the original objects and arrays. So primitive types and reference types differ significantly in the way they are assigned to variables, passed to methods, and copied. References in java are first class citizen.

The types also differ in the way they are compared for equality. When used with primitive values, the equality operator (==) simply tests whether two values are identical (i.e., whether they have exactly the same bits). With reference types, however, == compares references, not actual objects or arrays. In other words, == tests whether two references refer to the same object or array; it does not test whether two objects or arrays have the same content. For example:

```
String letter = "o";
String s = "hello"; // These two String objects
String t = "hell" + letter; // contain exactly the same text.
if (s == t) System.out.println("equal"); // But they are not equal!

byte[] a = { 1, 2, 3 }; // An array.
byte[] b = (byte[]) a.clone(); // A copy with identical content.
if (a == b) System.out.println("equal"); // But they are not equal!
```

Working with references has two kind of equality they are equality of references and equality of the objects they are referring. Every object in java has a method named equals() can be used to test the equality of two different objects. Two test objects for equivalence testing just pass one object as an argument to the equals method like below,

```
String letter = "o";
String s = "hello"; // These two String objects
String t = "hell" + letter; // contain exactly the same text.
if (s.equals(t)) // And the equals() method
    System.out.println("equivalent"); // tells us so.
```

Any class that wants to allow objects to be compared for equivalence testing can define its own version of the equals() method. One thing we should keep in mind that the default value for references is null.

Difference between Pointer and References:

Now I think I am at the point where I can explain the difference between pointers and references. We use both of these types of objects to access the memory however they differ in their way of action. So I think the main differences between pointers and references are:

1. A pointer is re-assignable:

```
int x = 5;
int y = 6;
```

```

int *p;
p = &x;
p = &y;
*p = 10;
assert(x == 5);
assert(y == 10);

```

In C++ a reference cannot, and must be assigned at initialization:

```

int x = 5;
int y = 6;
int &r = x;

```

Once it binds to object it cannot be rebind. However java references are re-assignable.

2. A pointer has its own memory address and size on the stack (4 bytes on x86) in C++, whereas a reference in C++ shares the same memory address (with the original variable) but also takes up some space on the stack. Since a reference has the same address as the original variable itself, it is safe to think of a reference as another name for the same variable. What a pointer points to can be on the stack or heap. What I am trying to say in this statement is not that a pointer must point to the stack. A pointer is just a variable that holds a memory address. This variable is on the stack. Since a reference has its own space on the stack, and since the address is the same as the variable it references. This implies that there is a real address of a reference that the compiler will not tell us. But in java reference acts like a C++ pointers. It has its own address in the stack and it will point something that will be stored at the heap only.

```

int x = 0;
int &r = x;
int *p = &x;
int *p2 = &r;
assert(p == p2);

```

3. We can have pointers to pointers to pointers offering extra levels of indirection. Whereas references only offer one level of indirection both in C++ and Java.

```

int x = 0;
int y = 0;
int *p = &x;
int *q = &y;
int **pp = &p;
pp = &q; /*pp = q
**pp = 4;
assert(y == 4);
assert(x == 0);

```

4. Pointer can be assigned NULL directly, whereas reference in C++ cannot. We can have a reference to a pointer, and then that pointer can contain NULL. In Java the default values for references is null.

```

int *p = NULL;
int &r = NULL; <--- compiling error

```

5. Pointers can iterate over an array, we can use ++ to go to the next item that a pointer is pointing to, and + 4 to go to the 5th element. This is no matter what size the object is that the pointer points to.

6. A pointer needs to be dereferenced with * to access the memory location it points to, whereas a reference can be used directly. A pointer to a class/struct uses -> to access its members whereas a reference uses a dot operator both in Java and C++.
7. A pointer is a variable that holds a memory address. Regardless of how a reference is implemented, a reference has the same memory address as the item it references in C++ which is not true in case of Java. References have separate address space in stack in Java.
8. Const references can be bound to temporaries. Pointers cannot (not without some indirection):

```
const int &x = int(12); //legal C++
int *y = &int(12); //illegal to dereference a temporary.
```

Memory Management:

The key to understanding the way memory works in any programming language is to understand what a variable is, and what its value is. At the most basic level, a variable is just an association between a name (used in the program's source code) and a slot of memory. A variable has a value, which is the content of the memory slot it's associated with. The size of that slot, and the interpretation of the value, depends on the type of the variable - and this is where the difference between value types and reference types comes in.

The value of a reference type variable is always either a reference or null. If it's a reference, it must be a reference to an object which is compatible with the type of the variable or can be null in Java. For instance, a variable declared as Stream s will always have a value which is either null or a reference to an instance of the Stream class. The slot of memory associated with the variable is just the size of a reference, however big the actual object it refers to might be. (On the 32-bit version of Java, for instance, a reference type variable's slot is always just 4 bytes.)

Where and how things are stored:

The memory slot for a variable is stored on either the stack or the heap. It depends on the context in which it is declared:

1. Each local variable (i.e. one declared in a method) is stored on the stack. That includes reference type variables - the variable itself is on the stack, but remember that the value of a reference type variable is only a reference (or null), not the object itself. Method parameters count as local variables too, but if they are declared with the ref modifier, they don't get their own slot, but share a slot with the variable used in the calling code.
2. Instance variables for a reference type are always on the heap. That's where the object itself "lives".
3. Instance variables for a value type are stored in the same context as the variable that declares the value type. The memory slot for the instance effectively contains the slots for each field within the instance. That means (given the previous two points) that a struct variable declared within a method will always be on the stack, whereas a struct variable which is an instance field of a class will be on the heap.
4. Every static variable is stored on the heap, regardless of whether it's declared within a reference type or a value type. There is only one slot in total no matter how many instances are created. (There don't need to be any instances created for that one slot to exist though.) The details of exactly which heap the variables live on are complicated.

Worked Example:

Here's a short program which does nothing useful, but should demonstrate the points we described above.

Codes written in java syntax,

```
class PairOfInts
{
    static int counter=0;

    public int a;
    public int b;

    public PairOfInts (int x, int y)
    {
        a=x;
        b=y;
        counter++;
    }
}

class Test
{
    PairOfInts pair;
    string name;

    public Test (PairOfInts p, string s, int x){
        pair = p;
        name = s;
        pair.a += x;
    }

    public static void Main(){
        PairOfInts z = new PairOfInts (1, 2);
        Test t1 = new Test(z, "first", 1);
        Test t2 = new Test(z, "second", 2);
        Test t3 = null;
        Test t4 = t1;
        // XXX
    }
}
```

Let's look at what's where in memory at the line marked with "XXX". (Assume that nothing is being garbage collected.)

1. There's a PairOfInts instance on the stack, corresponding with variable z. Within that instance, a=1 and b = 2.
2. There's a Test reference on the stack, corresponding with variable t1. This reference refers to an instance on the heap, which occupies "something like" 20 bytes: 8 bytes of header information (which all heap objects have), 8 bytes for the PairOfInts instance, and 4 bytes for the string reference. The value of the pair variable within that instance will have a=2 and b=2 . The value of the name variable within that instance will be a reference to a string object (which is also on the heap) and which (probably through other objects, such as a char array) represents the sequence of characters in the word "first".
3. There's a second Test reference on the stack, corresponding with variable t2. This reference refers to a second instance on the heap, which is very similar to the one described above, but with a reference to a

string representing "second" instead of "first", and with a value of pair where a=3 (as 2 has been added to the initial value 1). If PairOfInts were a reference type instead of a value type, there would only be one instance of it throughout the whole program, and just several references to the single instance, but as it is, there are several instances, each with different values inside.

4. There's a third Test reference on the stack, corresponding with variable t3. This reference is null - it doesn't refer to any instance of Test. (There's some ambiguity about whether this counts as a Test reference or not - it doesn't make any difference though, really - I generally think of null as being a reference which doesn't refer to any object, rather than being an absence of a reference in the first place. The Java Language Specification gives quite nice terminology, saying that a reference is either null or a pointer to an object of the appropriate type.)
5. There's a fourth Test reference on the stack, corresponding with variable t4. This reference refers to the same instance as t1. That is the values of t1 and t4 are the same. Changing the value of one of these variables would not change the value of the other, but changing a value within the object they both refer to using one reference would make that change visible via the other reference. (For instance, if we set t1.name="third"; then examined t4.name, we would find it referred to "third" as well.)
6. Finally, there's the PairOfInts.counter variable, which is on the heap (as its static). There's only a single "slot" for the variable, however many (or few) PairOfInts values there are.

Why two different storage:

Stack:

The stack is used as a sort of temporary scratch pad for use by the block of code that's currently executing, and whatever block called the current one, and whatever block called that one, and so on. When the current block exits, the local variables it was using are forgotten. As the name indicates, the stack is used in a last in, first out manner. One of the most important uses of the stack is to keep track of the current call chain. When one function calls another, the caller pushes the address of the next instruction (the return address) onto the stack. As each function exits, it pops it's caller's return address off the stack and continues executing code starting at that address. It's also used for communicating function parameters and return values between caller and callee. Freeing a block from the stack is nothing more than adjusting one pointer. It also allows us to implement the recursive function calls.

Heap:

The heap is memory set aside for dynamic allocation. Unlike the stack, there's no enforced pattern to the allocation and de allocation of blocks from the heap; we can allocate a block at any time and free it at any time. This makes it much more complex to keep track of which parts of the heap are allocated or free at any given time; there are many custom heap allocators available to tune heap performance for different usage patterns. Another thing about heap is if we want to allocate memory in a block of code and have that memory stick around beyond the end of the block, we allocate it on the heap. Of course, we will also need to store a pointer/reference to it somewhere so that other code can find that memory; most languages provide accommodation to that.

Allocation and De allocation speed:

Differences in speed aren't due to any property of the memory itself. Both stack and heap typically inhabit the same physical memory. Allocating space on the stack is quick due to the stacks LIFO nature: if we push something onto the stack, there's only one place it can end up. By contrast, allocating a block on the heap requires finding a large enough contiguous free region in memory. A stack allocation can be as quick as a single instruction; a heap allocation requires a call to a memory allocation function like malloc() in c or new keyword in C++ and java.

Static vs Dynamic nature:

Allocating memory on the heap is dynamic whether to allocate a block, and the size of the block, can be determined according to input the program receives while it's running. Regions of memory allocated on the heap can even be resized if necessary. It is possible to dynamically allocate memory on the stack too but that memory will be lost as soon as the current function exits. Stack allocations are usually static the compiler determines how much space is needed for the (non-register) parameters, return data, and local variables, and it generates code to reserve the necessary space on the stack when the function is called.

Final words on Stack and Heap:

Each thread gets a stack, while there's typically only one heap for the application (although it isn't uncommon to have multiple heaps for different types of allocation). The OS allocates the stack for each system-level thread when the thread is created. Typically the OS is called by the language runtime to allocate the heap for the application. The stack is attached to a thread, so when the thread exits the stack is reclaimed. The heap is typically allocated at application startup by the runtime, and is reclaimed when the application (technically process) exits. The size of the stack is set when a thread is created. The size of the heap is set on application startup, but can grow as space is needed (the allocator requests more memory from the operating system). The stack is faster because the access pattern makes it trivial to allocate and de allocate memory from it (a pointer/integer is simply incremented or decremented), while the heap has much more complex bookkeeping involved in an allocation or free. Also, each byte in the stack tends to be reused very frequently which means it tends to be mapped to the processor's cache, making it very fast. Another performance hit for the heap is that the heap, being mostly a global resource, typically has to be multi-threading safe, i.e. each allocation and de-allocation needs to be - typically - synchronized with "all" other heap accesses in the program.

So we have gained enough insight on stack and heap, two different kinds of allocation methods used by the language runtime. Now it is time to dissect how the memory management system works on these two.

Memory Management in C++:

C++ does not have any memory management system. If a programmer needs to use dynamic memory he has to deallocate the memory himself at the end of operation. One way to avoid this situation is to use as much stack memory as possible. Design the program in such a way so that program does not need to allocate dynamic memory. A properly written C++ program should have minimum amount of dynamic memory usage. There is another way to manage memory in C++ and that is RAII. Resource Acquisition Is Initialization is the technique was developed for exception-safe resource management in C++.

In RAII, holding a resource is tied to object lifetime, resource allocation (acquisition) is done during object creation (specifically initialization), by the constructor, while resource de-allocation (release) is done during object destruction, by the destructor. If objects are destructed properly, resource leaks do not occur. The RAII design is often used for controlling mutex locks in multi-threaded applications. In that use, the object releases the lock when destroyed. Without RAII in this scenario the potential for deadlock would be high and the logic to lock the mutex would be far from the logic to unlock it. With RAII, the code that locks the mutex essentially includes the logic that the lock will be released when execution leaves the scope of the RAII object. The advantages of RAII as a resource management technique are that it provides encapsulation, exception safety (for stack resources), and locality (it allows acquisition and release logic to be written next to each other). Encapsulation is provided because resource management logic is defined once in the class, not at each call site. Exception safety is provided for stack resources (resources that are released in the same scope as they are acquired) by tying holding the resource to the lifetime of a stack variable (a local variable declared in a given scope): if an exception is thrown, and proper exception handling

is in place, the only code that will be executed when exiting the current are the destructors of objects declared in that scope. Finally, locality of definition is provided by writing the constructor and destructor definitions next to each other in the class definition.

Memory Management in Java:

In Java objects reside in an area called the heap. The heap is created when the JVM starts up and may increase or decrease in size while the application runs. When the heap becomes full, *garbage is collected*. During the garbage collection objects that are no longer used are cleared, thus making space for new objects.

Note that the JVM uses more memory than just the heap. For example Java methods, thread stacks and native handles are allocated in memory separate from the heap, as well as JVM internal data structures.

The heap is sometimes divided into two areas (or generations) called the nursery (or young space) and the old space. The nursery is a part of the heap reserved for allocation of new objects. When the nursery becomes full, garbage is collected by running a special young collection, where all objects that have lived long enough in the nursery are promoted (moved) to the old space, thus freeing up the nursery for more object allocation. When the old space becomes full garbage is collected there, a process called an old collection.

The reasoning behind a nursery is that most objects are temporary and short lived. A young collection is designed to be swift at finding newly allocated objects that are still alive and moving them away from the nursery. Typically, a young collection frees a given amount of memory much faster than an old collection or a garbage collection of a single-generational heap (a heap without a nursery).

During object allocation, the JRockit JVM distinguishes between *small* and *large* objects. The limit for when an object is considered large depends on the JVM version, the heap size, the garbage collection strategy and the platform used, but is usually somewhere between 2 and 128 kB. Small objects are allocated in thread local areas (TLAs). The thread local areas are free chunks reserved from the heap and given to a Java thread for exclusive use. Large objects that don't fit inside a TLA are allocated directly on the heap. When a nursery is used, the large objects are allocated directly in old space.

Now it is time to describe about the final thing garbage collection. Garbage collection is the process of freeing space in the heap or the nursery for allocation of new objects. This section describes the garbage collection used in the JRockit JVM. JVM uses several garbage collection models to handle different situation of heap memory state. They are:

1. The Mark and Sweep Model
2. Generational Garbage Collection
3. Dynamic and Static Garbage Collection Modes
4. Compaction

I am going to describe each of them in a few words.

The Mark and Sweep Model:

The JRockit JVM uses the mark and sweep garbage collection model for performing garbage collections of the whole heap. A mark and sweep garbage collection consists of two phases, the mark phase and the sweep phase.

During the mark phase all objects that are reachable from Java threads, native handles and other root sources are marked as alive, as well as the objects that are reachable from these objects and so forth. This process identifies and marks all objects that are still used, and the rest can be considered garbage. During the sweep phase the heap is

traversed to find the gaps between the live objects. These gaps are recorded in a free list and are made available for new object allocation.

General Garbage Collection:

The nursery, when it exists, is garbage collected with a special garbage collection called a young collection. A garbage collection strategy which uses a nursery is called a generational garbage collection strategy, or simply generational garbage collection. The young collector used in the JRockit JVM identifies and promotes all live objects in the nursery that are outside the keep area to the old space. This work is done in parallel using all available CPUs. The Java threads are paused during the entire young collection.

Dynamic and Static Garbage Collection Mode:

By default, the JRockit JVM uses a dynamic garbage collection mode that automatically selects a garbage collection strategy to use, aiming at optimizing the application throughput. The dynamic modes available are:

- Throughput, which optimizes the garbage collector for maximum application throughput. This is the default mode.
- Pausetime, which optimizes the garbage collector for short and even pause times for threads and application.
- Deterministic, which optimizes the garbage collector for very short and deterministic pause times.

Compaction:

Objects that are allocated next to each other will not necessarily become unreachable (die) at the same time. This means that the heap may become fragmented after a garbage collection, so that the free spaces in the heap are many but small, making allocation of large objects hard or even impossible. Free spaces that are smaller than the minimum thread local area (TLA) size cannot be used at all, and the garbage collector discards them as dark matter until a future garbage collection frees enough space next to them to create a space large enough for a TLA. To reduce fragmentation, the JRockit JVM compacts a part of the heap at every garbage collection (old collection). Compaction moves objects closer together and further down in the heap, thus creating larger free areas near the top of the heap. The size and position of the compaction area as well as the compaction method is selected by advanced heuristics, depending on the garbage collection mode used. Compaction is performed at the beginning of or during the sweep phase and while all Java threads are paused.

Conclusion:

There are lot things over pointers, references, memory management and garbage collection can be described. I just tried to give an overview on the whole model. Near future I will work more on this to gain further insight in this matter.